

Portfolio Website — Build Roadmap

Rishi Chaudhari | Premium Personal Portfolio

A step-by-step execution plan, rewritten from the original PRD into copy-paste-ready prompts for an AI coding agent (Claude Code, Cursor, Windsurf, Bolt, v0, etc.), with gaps filled in so the agent never has to guess.

Tech Stack	Next.js 14 (App Router) + TypeScript + Tailwind + shadcn/ui + Framer Motion
Design Language	Apple × Stripe × Linear — minimal, whitespace-driven, precise
Total Phases	11 (Phase 0 → Phase 10), each with a ready-to-paste agent prompt
How to use this doc	Work top to bottom. Paste each 'AGENT PROMPT' box into your coding agent. Don't skip Phase 0

What I Changed vs. Your Original PRD

Your original PRD was strong on vision and design language. Here's what was missing before it's actually usable as an execution plan for a coding agent — an agent can't invent your resume, photos, or project metrics, so those gaps had to be closed first.

- Added Phase 0 (Content & Asset Collection). This is the #1 reason AI-built portfolios stall halfway — the agent hits a wall needing real copy, photos, and project write-ups that don't exist yet. This is now a checklist you fill in BEFORE writing any prompt.
- Split the old 'Projects' phase into two: (5a) build the reusable case-study template, (5b) populate it with your actual two projects. Mixing template-building and content-writing in one prompt makes agents produce inconsistent, half-filled sections.
- Added a real Contact Form backend step (Resend or EmailJS) — a static Next.js site can't send emails without one, and the original PRD left this unresolved.
- Made content editable without redeploying code: project case studies, achievements, and certifications now live in structured JSON/MDX files instead of hardcoded JSX, so you can add a new hackathon win without asking the agent to touch component code again.
- Added an explicit Repo & Deployment phase (Phase 1) instead of leaving Vercel as an afterthought — env vars, preview deployments, and domain setup are set up early so every later phase can be checked live.
- Added a QA / Lighthouse / Accessibility pass as its own phase with a concrete checklist, instead of a vague 'Polish' step.
- Trimmed 'Future Features' to a real backlog (Section at the end) so the agent doesn't try to build a Spotify widget before your Experience section exists.

■ *Everything from your original PRD (color palette, typography, tech stack, section list) is preserved — nothing was removed, only reorganized and made buildable.*

How To Use This Document

Each phase below has three parts:

- Goal — what this phase accomplishes and why it comes at this point in the sequence.
- Prerequisites — what must exist before you run this phase's prompt (files, accounts, content).
- AGENT PROMPT — copy this entire block and paste it as-is into Claude Code, Cursor, or your agent of choice. Fill in the [BRACKETED] placeholders with your real info first.

Run phases strictly in order. Each prompt assumes the previous phase's output already exists in the repo. After each phase, review the agent's output, commit it to git, and only then move to the next prompt — don't batch multiple phases into one giant prompt, it degrades output quality and makes debugging much harder.

Phase 0: Content & Asset Collection (Do This Before Any Code)

Goal: Gather every real piece of content the site needs, so the agent is never guessing or inventing placeholder text that you'll have to rip out later.

This phase has no agent prompt — it's a checklist for you. Save everything into one folder, e.g. /content-source/, before Phase 1.

Checklist

- Professional headshot (min. 1200×1200px) + 1-2 casual/leadership photos for the About/Leadership sections
- ATS-friendly resume PDF + a designed one if you want two versions
- Final bio copy for Hero and About (2-3 sentence hero subtitle, 150-200 word About paragraph)
- For each project (Retail AI Platform, AD² Restormer, + any future ones): problem statement, objectives, business need, tech stack list, 3-6 screenshots, architecture diagram image (or rough sketch you can ask the agent to turn into an SVG diagram), GitHub link, live demo link if any, 2-3 measurable outcomes/metrics, key challenges + how you solved them
- Leadership roles (GDG, Robofiesta, Theatre Club, MUN): role title, duration, team size, 2-3 impact bullet points each, 2-4 photos each
- Experience entries: organization, role, duration, 2-4 responsibility/impact bullets each
- Full skills list grouped by category (Programming, Backend, Frontend, AI, Analytics, Cloud, DevOps, Business, Soft Skills)
- Achievement/hackathon entries: competition name, award, 1-2 line description, certificate image, photo, date
- Certification badges/images (Microsoft, Google, DeepLearning.AI, Scrum, Great Learning) + verification links
- Contact details: email, phone (optional), LinkedIn, GitHub, city/location
- Domain name decision (e.g. rishichaudhari.dev) and confirm you have a Vercel account

Phase 1: Repo, Foundation & Deployment Pipeline

Goal: Stand up the project skeleton and get a live preview URL working on day one, so every future phase can be checked in production, not just localhost.

Prerequisites: A GitHub account, a Vercel account, Node.js installed locally.

AGENT PROMPT — Phase 1

```
You are setting up the foundation for a premium personal portfolio website.

Tech stack: Next.js 14 (App Router), TypeScript, Tailwind CSS, shadcn/ui, Framer Motion,
Lucide icons. Deployment target: Vercel.

Tasks:
1. Initialize a new Next.js 14 project with TypeScript, App Router, and Tailwind CSS
   pre-configured (use create-next-app with the correct flags).
2. Install and configure shadcn/ui, Framer Motion, and lucide-react.
3. Set up ESLint + Prettier with sensible defaults for a production project.
4. Create a .env.example file with placeholders for any keys we'll need later
   (RESEND_API_KEY, NEXT_PUBLIC_SITE_URL).
5. Set up the folder structure:
/app
/components (ui/, sections/, shared/)
/content (JSON or MDX files for projects, achievements, certifications, experience,
leadership - do NOT hardcode this data inside components)
/lib (utils, constants)
/public (images, resume PDF, favicon)
6. Configure next/font with Space Grotesk (headings), Inter (body), and JetBrains Mono
   (code snippets only).
7. Add a ThemeProvider for light/dark mode using next-themes, defaulting to light mode.
8. Set up basic SEO defaults in the root layout (title template, meta description
   placeholder, favicon placeholder).
9. Initialize git, commit, push to a new GitHub repo, and connect it to Vercel so we get
   a live preview URL.
10. Confirm the site builds and deploys with just a placeholder "Hello" page before
    moving on.

Output: a working, deployed Next.js project with the folder structure above and a live
Vercel URL. Do not build any real sections yet - that's the next phase.
```

Phase 2: Design System & Reusable Components

Goal: Build the component library and design tokens once, so every later section reuses the same building blocks instead of one-off styles.

Prerequisites: Phase 1 complete and deployed.

AGENT PROMPT — Phase 2

Build the design system for the portfolio site on top of the Phase 1 foundation.

Design tokens (add as Tailwind theme extensions / CSS variables):

- Background: #FAFAFA (light) / #0B1120 (dark)
- Card background: #FFFFFF (light) / a slightly lighter dark surface for dark mode
- Primary text: #111827 (light) / near-white for dark
- Secondary text: #6B7280
- Accent: #2563EB
- Success: #14B8A6
- Headings font: Space Grotesk, weight 700
- Body font: Inter, weight 400
- Monospace font: JetBrains Mono (code snippets only)
- Border radius: consistently rounded (use a single --radius token)
- Shadows: soft, subtle only – no heavy drop shadows
- Spacing: generous whitespace, define a consistent section vertical rhythm (e.g. py-24 on desktop, py-16 on mobile)

Build these reusable components using shadcn/ui as the base where sensible:

1. Navbar – sticky, transparent at top, blurs and adds a background + border on scroll, includes theme toggle and a "Resume" button
2. Footer – minimal, "Designed & Developed by [Name] © 2026"
3. Button (primary, secondary, ghost variants)
4. Card (generic content card)
5. Section (wrapper component enforcing consistent max-width and vertical padding)
6. Timeline (for Experience/Leadership/Education – vertical line with dot markers)
7. Badge (for skills, certifications, tech stack pills)
8. Modal/Dialog (for expanded project screenshots)
9. ProjectCard (image, title, short description, tags, "View case study" link)
10. AchievementCard (competition, award, date, thumbnail)
11. StatCard (big number + label, used in About stats)

Animation rules (Framer Motion):

- Allowed: fade, slide, scale, blur/opacity transitions
- Not allowed: bounce, rotate, parallax, floating objects
- Respect `prefers-reduced-motion` – disable/simplify animations for users who have this set
- Animations should feel instant and purposeful, not decorative

Output: a component library visible on a temporary `/design-system` page showing every component and its variants, so we can visually review it before building real pages.

Phase 3: Home Page — Hero & About Preview

Goal: Build the first thing every recruiter sees. This page alone should communicate the whole positioning in under 5 seconds of scanning.

Prerequisites: Phase 2 components approved. Real headshot photo and bio copy from Phase 0 ready in /public and /content.

AGENT PROMPT — Phase 3

Build the Home page hero and about-preview sections using the design system from Phase 2.

Hero section:

- Large open whitespace, professional photo of [NAME] (use /public/images/headshot.jpg)
- Headline: "Hi, I'm [FULL NAME]."
- Subtitle: "[FINAL HERO SUBTITLE FROM CONTENT COLLECTION]"
- CTA buttons: "View Projects" (scrolls to/links to Projects), "Download Resume" (links to /resume.pdf), GitHub icon link, LinkedIn icon link
- Background: a very subtle radial gradient using the accent color at low opacity – no busy patterns, no floating shapes
- Entrance animation: simple fade + slight upward slide on load, staggered for headline → subtitle → CTAs

About preview section (short version – full About page comes in Phase 4):

- 2-3 sentence professional intro pulled from /content/about.json
- Row of StatCards: Projects count, Leadership roles count, Hackathons count, Certifications count (pull these numbers from the content files, don't hardcode)
- "Read more about me" link to the full About page/section

Make the whole page fully responsive (mobile, tablet, desktop, ultra-wide) and verify it looks correct at 375px, 768px, 1280px, and 1920px widths.

Output: a live Home page on the Vercel preview URL with working hero and about-preview sections, using only real content from Phase 0 – no lorem ipsum, no placeholder photos.

Phase 4: Core Content Sections

Goal: Build About, Experience, Leadership, Skills, Achievements, and Certifications — the sections that do the heavy lifting of proving credibility.

Prerequisites: Phase 0 content files for experience, leadership, skills, achievements, certifications are filled in as JSON under /content.

AGENT PROMPT — Phase 4

Build the following sections/pages, each reading from its corresponding JSON file in /content (do not hardcode data in the components):

1. About – full bio, education timeline, career-journey timeline (use the Timeline component from Phase 2)
2. Experience – professional timeline using /content/experience.json. Each entry: organization, role, duration, description, skills used, optional media/logo
3. Leadership – timeline/cards using /content/leadership.json for GDG, Robofiesta, Theatre Club, MUN. Each entry: role, impact bullets, team size, achievements, photo gallery (use the Modal component for expanded gallery view)
4. Skills – categorized grid using /content/skills.json, grouped into Programming, Backend, Frontend, AI, Analytics, Cloud, DevOps, Business, Soft Skills. Each skill shows an icon (lucide-react or simple-icons) + name.
5. Achievements – grid of AchievementCard components using /content/achievements.json. Clicking a card opens a modal with the certificate image, full description, and date.
6. Certifications – badge grid using /content/certifications.json (Microsoft, Google, DeepLearning.AI, Scrum, Great Learning), each badge links to its verification URL.

All sections must:

- Use the Section wrapper for consistent spacing
- Animate into view on scroll (fade + slight slide, once per element, not repeating)
- Be fully responsive
- Have real content only – pull directly from the JSON files created in Phase 0

Output: all six sections live and navigable from the Navbar, each populated with real content, no placeholder data.

Phase 5a: Reusable Project Case-Study Template

Goal: Build ONE flexible template component that any project can plug into — build the shape first, fill it with real content in the next phase.

Prerequisites: Phase 4 complete.

AGENT PROMPT — Phase 5a

Build a reusable Project Case Study template (a dynamic route, e.g. `/projects/[slug]`) that renders from a project data shape. Do not hardcode any specific project's content in this phase — this is purely the template/component structure.

Define a TypeScript type/interface for a project with these fields:

- slug, title, heroImage, category (AI / Analytics / Full Stack / Leadership)
- problem, objectives, businessNeed (each a short rich-text/markdown block)
- architectureDiagram (image path)
- techStack (array of {name, icon})
- features (array of strings or {title, description})
- challenges (array of {challenge, solution} pairs)
- metrics (array of {label, value} — e.g. "40% faster inference")
- lessonsLearned (rich text block)
- screenshots (array of image paths, shown in a gallery/carousel)
- githubUrl, demoUrl (optional)
- futureImprovements (array of strings)

Build the case-study page layout in this section order: Hero Image → Problem → Objectives → Business Need → Architecture Diagram → Tech Stack (badge row) → Features → Challenges & Solutions (side-by-side or accordion) → Metrics (StatCard row) → Screenshots gallery (lightbox on click) → Lessons Learned → Future Improvements → GitHub/Demo buttons.

Also build the Projects listing page/section with a filterable grid (filter by category: AI, Analytics, Full Stack, Leadership) using ProjectCard components that link into the case-study template.

Use one dummy project with obviously-fake placeholder text (clearly marked "PLACEHOLDER — replace in Phase 5b") just to verify the template renders correctly end-to-end. Do not write real project content in this phase.

Output: a working, styled case-study template and a filterable project listing page, verified with one placeholder project.

Phase 5b: Populate Real Project Content

Goal: Fill the template from Phase 5a with your two real projects (and leave it easy to add more later).

Prerequisites: Phase 5a template working. Real project content from Phase 0 (Retail AI Platform, AD² Restormer) ready.

AGENT PROMPT — Phase 5b

Populate the project case-study template from Phase 5a with real content for two projects. Create `/content/projects/retail-ai-platform.json` and `/content/projects/ad2-restormer.json` (or `.mdx` if richer formatting is needed for the problem/lessons sections) matching the TypeScript interface already defined.

Project 1 – Retail AI Platform:

[PASTE: problem, objectives, business need, tech stack, features, challenges + solutions, metrics, screenshots list, GitHub link, demo link, future improvements from your Phase 0 content collection]

Project 2 – AD² Restormer:

[PASTE: same fields as above for this project]

Remove the placeholder/dummy project from Phase 5a once both real projects render correctly. Verify:

- Both case studies render fully with no leftover placeholder/lorem ipsum text
- All images load correctly and are optimized via `next/image`
- The category filter on the Projects listing page correctly shows/hides these two
- GitHub and demo links open in a new tab and are correct

Output: two fully populated, real project case studies live on the site, listing page filter working correctly.

Phase 6: Contact, Resume & Email Delivery

Goal: Make the site actually functional for a recruiter to reach out or download your resume — not just look good.

Prerequisites: Resend or EmailJS account created. Final resume PDF placed in /public/resume.pdf.

AGENT PROMPT — Phase 6

Build the Contact section and Resume delivery.

Contact section:

- Minimal layout with Email, LinkedIn, GitHub, optional Phone, and Location text
- A contact form (Name, Email, Message) with client-side validation (required fields, valid email format) using a schema validation library (zod is fine)
- Wire the form to a server action / API route that sends the message via Resend (use RESEND_API_KEY from .env — do not hardcode any key in source)
- Show a clear success state and a clear error state in the UI after submission
- Add basic spam protection (honeypot field is sufficient, no need for a CAPTCHA service unless spam becomes a real problem)

Resume section:

- Embed a PDF viewer showing /public/resume.pdf inline
- Prominent one-click "Download Resume" button
- Confirm the PDF is the ATS-friendly version, not a heavily designed one, unless a separate ATS version file is also provided — if two versions exist, offer both with clear labels ("ATS Version" / "Designed Version")

Output: a working contact form that successfully delivers a real test email, and a resume section that correctly displays and downloads the real PDF.

Phase 7: SEO, Metadata & Structured Data

Goal: Make sure the site is actually discoverable and shares well when linked on LinkedIn/Twitter/WhatsApp.

AGENT PROMPT — Phase 7

Add complete SEO and sharing metadata to the site.

1. Set a real meta title and meta description per page (Home, About, Projects, Experience, Leadership, Achievements, Contact) – not generic Next.js defaults.
2. Generate an Open Graph image (1200x630) with name, title, and a clean background matching the site's design language – this can be a static image or generated dynamically with next/og.
3. Add Twitter Card metadata using the same OG image.
4. Add JSON-LD structured data using the Person schema (name, jobTitle, url, sameAs for LinkedIn/GitHub).
5. Generate robots.txt and sitemap.xml (use next-sitemap or the App Router's built-in sitemap.ts).
6. Add a real favicon set (favicon.ico, apple-touch-icon, and manifest icons).
7. Add a custom 404 page matching the site's design language, with a link back home.

Output: correct metadata verified by testing the site URL through a social share debugger (e.g. paste the live URL into a Twitter Card validator or LinkedIn Post Inspector) and confirming the OG image and description render correctly.

Phase 8: Accessibility & Responsive QA

Goal: Confirm the site is usable by everyone and on every screen size — not just visually reviewed by you on one laptop.

AGENT PROMPT — Phase 8

Run a full accessibility and responsiveness pass across the entire site.

Accessibility:

1. Verify full keyboard navigation works (tab order is logical, all interactive elements are reachable and have visible focus states)
2. Add proper ARIA labels to icon-only buttons (theme toggle, social links, modal close buttons, carousel controls)
3. Verify color contrast meets WCAG AA on both light and dark mode for all text/background combinations, especially secondary text (#6B7280) on white/dark
4. Add meaningful alt text to every image (headshot, project screenshots, certificates, gallery photos) – not generic "image" alt text
5. Confirm `prefers-reduced-motion` correctly disables/simplifies all Framer Motion animations site-wide
6. Run an automated audit (axe DevTools or Lighthouse accessibility score) and fix every flagged issue

Responsiveness:

1. Test and fix layout at these exact breakpoints: 375px (mobile), 768px (tablet), 1280px (laptop), 1920px (desktop/ultra-wide)
2. Confirm the Navbar collapses to a mobile menu below 768px with no layout shift
3. Confirm image galleries/modals work correctly with touch gestures on mobile
4. Confirm the contact form is usable and correctly validated on mobile

Output: a written summary of every issue found and fixed, plus final Lighthouse Accessibility and Best Practices scores.

Phase 9: Performance Optimization

Goal: Hit the Lighthouse targets from the original PRD — this is what actually separates 'looks premium' from 'is premium'.

AGENT PROMPT — Phase 9

Optimize the site for Lighthouse performance, aiming for 90+ (target 100 where realistic) on Performance, SEO, Accessibility, and Best Practices.

1. Audit and optimize every image: correct sizing via next/image, modern formats (WebP/AVIF), lazy-loading below the fold, priority loading for the hero image only
2. Audit bundle size: check for unused shadcn/ui components, unused Framer Motion features, and unnecessary client-side JavaScript – convert components to Server Components wherever they don't need interactivity
3. Add loading states/skeletons for any content that loads client-side (e.g. contact form submission, image galleries)
4. Add smooth but lightweight page transitions – verify they don't hurt Core Web Vitals (particularly CLS and INP)
5. Preload critical fonts, confirm no layout shift from font loading (use next/font's built-in optimization, verify with a real Lighthouse run rather than assuming)
6. Run Lighthouse (mobile + desktop) on the deployed Vercel URL, not localhost, and report the four scores plus any remaining warnings

Output: a before/after Lighthouse score comparison and a list of any remaining warnings that were consciously deprioritized (with reasoning).

Phase 10: Final Deployment & Launch Checklist

Goal: Ship it — with a custom domain, and a final human review pass before sharing the link with recruiters.

AGENT PROMPT — Phase 10

Prepare the site for final launch.

1. Connect the custom domain [YOUR DOMAIN] to the Vercel project, confirm SSL is active and the www/non-www redirect is configured correctly
2. Set all production environment variables in Vercel (RESEND_API_KEY, NEXT_PUBLIC_SITE_URL) and confirm the contact form works on the production domain, not just the preview URL
3. Do a final content pass: search the entire codebase for "lorem", "placeholder", "TODO", "[BRACKET]" and confirm nothing was missed
4. Confirm the resume PDF, all project links, and all certification verification links are correct and open properly
5. Re-run Lighthouse one final time on the production domain and confirm scores hold
6. Verify social share previews (OG image, description) render correctly when the production URL is shared

Output: a live, production-ready portfolio at [YOUR DOMAIN] with a final checklist confirming every item above is complete.

Appendix A — Future Features Backlog

Don't build these until the core site (Phases 0-10) is live and shared with real recruiters. Ship first, iterate after.

- Blog / writing section (MDX-based, reuse the case-study content pattern)
- Visitor analytics dashboard (Vercel Analytics or Plausible — privacy-friendly, no cookie banner needed)
- GitHub API widget (pinned repos, contribution graph)
- LeetCode/competitive-programming stats widget
- Project search/command-palette (cmdk)
- Spotify 'now playing' widget

Appendix B — Design Reference (from original PRD)

Palette	Bg #FAFAFA / Dark #0B1120 · Card #FFFFFF · Text #111827 · Secondary #6B7280 · Accent #2563EB · Su
Typography	Headings: Space Grotesk 700 · Body: Inter 400 · Code: JetBrains Mono
Animation	Allowed: fade, slide, scale, blur/opacity. Avoid: bounce, rotate, parallax, floating objects
Inspiration	Apple, Stripe, Linear, Vercel, OpenAI, Notion
Guiding line	"If Apple were hiring an AI engineer with strong consulting and leadership experience, this is the portfolio they